Lecture 7: Pipelining & Performance Enhancement

1. Concept of Pipelining (Instruction Pipeline Stages)

In the Fetch-Decode-Execute cycle, the CPU executes one instruction completely before starting the next. This is like a single-lane road where only one car can pass at a time. **Pipelining** is an architectural technique that allows multiple instructions to be in different stages of execution simultaneously, much like an assembly line.

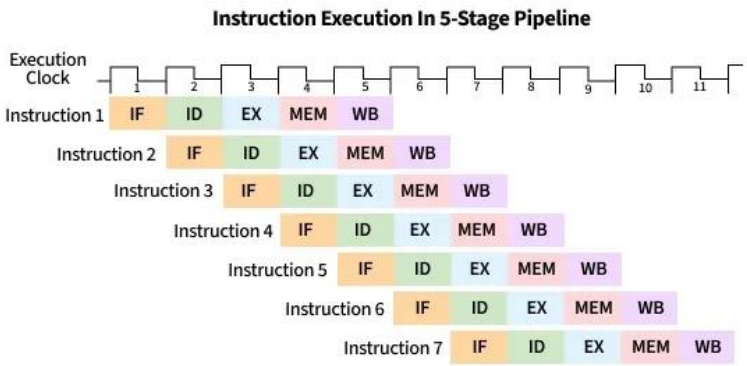
The basic idea is to break down the instruction execution cycle into smaller, sequential stages. When one instruction completes a stage, it moves to the next, and the next instruction can immediately enter the now-free stage.

1.1. Typical Instruction Pipeline Stages (e.g., 5-stage pipeline):

1. **Instruction Fetch (IF):** Fetch the instruction from memory (PC points to it).
2. **Instruction Decode (ID):** Decode the instruction, read operands from registers.
3. **Execute (EX):** Perform the ALU operation (arithmetic, logical) or calculate the effective address for memory operations.
4. **Memory Access (MEM):** Access memory (read data for LOAD, write data for STORE).
5. **Write Back (WB):** Write the result back to a register.

How it works:

- At any given clock cycle, each stage can be working on a different instruction.
- After the initial "fill-up" time (equal to the number of stages), one instruction completes execution *per clock cycle* (ideally).
- This greatly increases the **throughput** (instructions completed per unit of time), even if the **latency** (time for a single instruction to complete) remains the same or slightly increases.



Analogy: A laundry analogy.

- **Non-pipelined:** Wash (30min) -> Dry (40min) -> Fold (20min). Total 90min per load. Can't start next load until previous is completely folded.
- **Pipelined:** You have separate machines.
    - Load 1: Wash (30min)
    - Load 2: Wash (30min) | Load 1: Dry (40min)
    - Load 3: Wash (30min) | Load 2: Dry (40min) | Load 1: Fold (20min)
    - After the initial 90 min (for Load 1), a new load of laundry is completed every 30 minutes (the time of the longest stage).

2. Pipeline Hazards and Basic Solutions

Pipelining works best when instructions flow smoothly. However, certain conditions can disrupt this flow, leading to **pipeline hazards**, which cause stalls (delays) in the pipeline.

2.1. Types of Hazards:

1. **Structural Hazards (Resource Conflicts):**
    - Occur when two or more instructions simultaneously try to access the same hardware resource.
    - **Example:** If there's only one memory unit, and one instruction needs to fetch an instruction (IF stage) while another needs to access data from memory (MEM stage) in the same clock cycle.
    - **Basic Solution:**
        - **Stalling/Bubbles:** Temporarily halt the instruction that needs the resource until it becomes available. This is like inserting "bubbles" (NOP operations) into the pipeline.
        - **Separate Resources:** Provide separate hardware units (e.g., separate instruction and data caches/memory – a form of Harvard architecture).
2. **Data Hazards (Data Dependencies):**
    - Occur when an instruction needs data that has not yet been produced by a preceding instruction in the pipeline.

- **Example:** ADD R1, R2, R3 (R1 = R2 + R3) SUB R4, R1, R5 (R4 = R1 - R5) The SUB instruction needs the new value of R1 (produced by ADD) which might still be in the ALU or WB stage of the ADD instruction.
- **Basic Solutions:**
  - **Stalling/Bubbles:** Stall the dependent instruction until the data is available.
  - **Forwarding/Bypassing:** The most common solution. The result of an operation is forwarded directly from the ALU's output (or a later stage) back to the input of an earlier stage where it's needed, *before* it's written back to the register file. This avoids a stall.
  - **Compiler Scheduling:** Compilers can reorder instructions to avoid data hazards (if dependencies allow) or insert NOPs.

3. **Control Hazards (Branch Hazards):**
   - Occur when the pipeline makes a wrong assumption about which instruction to fetch next due to a branch instruction.
   - **Example:** A JUMP or BRANCH_IF_ZERO instruction changes the PC, but instructions after the branch instruction are already fetched into the pipeline. If the branch is taken, these fetched instructions are wrong ("speculative" execution).
   - **Basic Solutions:**
     - **Stalling/Flushing:** Stall the pipeline until the branch target is known, then flush (discard) any wrongly fetched instructions and fetch from the correct target.
     - **Branch Prediction:** The CPU attempts to predict whether a branch will be taken or not. If the prediction is correct, the pipeline continues smoothly. If incorrect, a penalty (flush and refill) occurs.
     - **Delayed Branch:** The instruction *after* the branch instruction is *always* executed, regardless of whether the branch is taken. The compiler places a useful instruction there or a NOP. (Less common in modern complex pipelines).

3. Introduction to Parallelism (Flynn's Taxonomy - Brief)

Beyond pipelining, computer architectures employ various forms of parallelism to achieve even higher performance by doing multiple things simultaneously. Flynn's Taxonomy categorizes computer architectures based on the number of instruction streams and data streams they can handle concurrently.

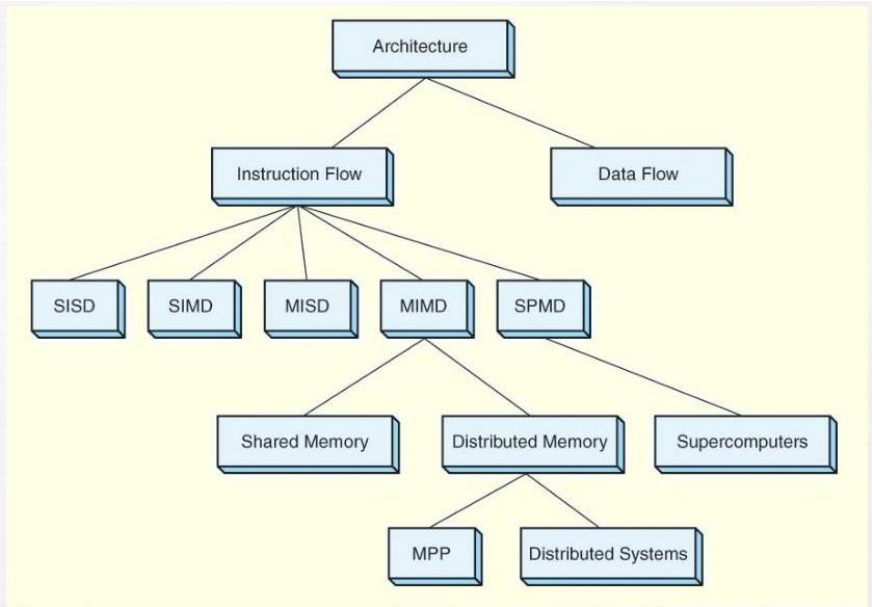


Figure 1 – Flynn's Taxonomy

- **SISD (Single Instruction, Single Data):**
  - Traditional uniprocessor systems.
  - One instruction stream operates on one data stream at a time.
  - Pipelining can improve performance, but it's still fundamentally SISD.
  - **Example:** Classic Von Neumann architecture, single-core CPUs without specific parallel features.
- **SIMD (Single Instruction, Multiple Data):**
  - A single instruction operates simultaneously on multiple data elements.
  - Excellent for tasks that involve repetitive operations on large sets of data (e.g., graphics processing, multimedia, scientific computing).
  - **Example:** Vector processors, modern GPUs (Graphics Processing Units), CPU extensions like Intel's SSE/AVX or ARM's NEON.
- **MISD (Multiple Instruction, Single Data):**
  - Multiple instructions operate on a single data stream.
  - Very rare in practice, primarily for fault-tolerant systems where multiple processors perform the same task and compare results.

    - **Example:** Specialized redundant systems.
- **MIMD (Multiple Instruction, Multiple Data):**
    - Multiple independent instruction streams operate on multiple independent data streams.
    - This is the most common form of true parallel computing.
    - **Example:** Multi-core processors (each core is an independent processor), multi-processor systems (multiple CPUs), clusters, supercomputers. This is what you typically think of when you hear "parallel computing."

4. Performance Metrics

To evaluate and compare computer systems, various performance metrics are used:

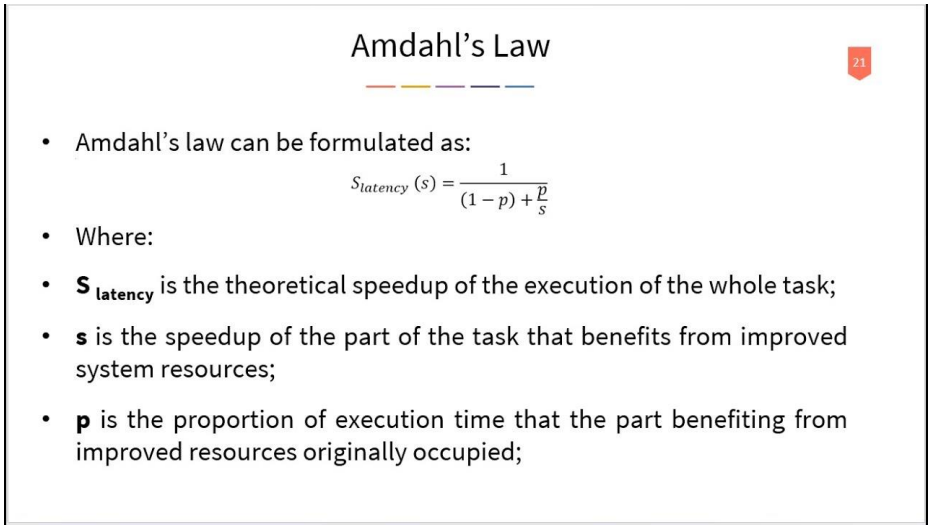
- **Clock Speed (Clock Rate):**
    - Measured in Hertz (Hz), typically Gigahertz (GHz).
    - Indicates how many clock cycles the CPU completes per second.
    - A higher clock speed generally means faster execution, but it's not the only factor. A 4 GHz CPU doesn't automatically mean it's twice as fast as a 2 GHz CPU because of CPI and ISA differences.
- **Cycles Per Instruction (CPI):**
    - The average number of clock cycles required to execute one instruction.
    - **Lower CPI is better.**
    - Pipelining aims to achieve a CPI close to 1 (one instruction completing per cycle).
    - CISC architectures tend to have higher CPI than RISC architectures (for their more complex instructions).
- **MIPS (Millions of Instructions Per Second):**
    - A common metric for measuring processor speed.
    - MIPS = (Clock Rate / CPI) / 1,000,000
    - **Limitations:** Can be misleading because different ISAs have different instructions (a "simple" instruction on RISC might take multiple "complex" CISC instructions). Thus, comparing MIPS across different ISAs is often not fair.

- **FLOPS (Floating-point Operations Per Second):**
    - Measures the number of floating-point arithmetic operations (additions, multiplications) a processor can perform per second.
    - **Higher FLOPS is better.**
    - Crucial for scientific computing, graphics, and machine learning, where floating-point calculations are dominant.
    - Often expressed in MegaFLOPS (MFLOPS), GigaFLOPS (GFLOPS), TeraFLOPS (TFLOPS), PetaFLOPS (PFLOPS).
- **Execution Time:** The ultimate measure of performance. It's how long a program takes to run.
    - **Execution Time = (Number of Instructions) × CPI × (Clock Cycle Time)**
    - Or, **Execution Time = (Number of Instructions × CPI) / Clock Rate**

5. Amdahl's Law (Basic Understanding)

Amdahl's Law is a formula that provides an upper bound on the speedup that can be achieved by parallelizing a portion of a program. It states that the performance improvement of parallel computing is limited by the sequential portion of the program.

- **Formula:** Slatency(s)=(1−P)+sP1



    - (1−P): the proportion of execution time that the part *not* benefiting from improvement originally occupied (the sequential portion).
- **Key Insight:** Even if you make the parallelizable part infinitely fast (s→∞), the total speedup is still limited by the sequential part.
    - If 10% of a program is inherently sequential (P=0.9), and the remaining 90% can be perfectly parallelized (s=∞), the maximum speedup is

$1/(1-0.9)=1/0.1=10\times$. You can never achieve more than 10 times speedup, no matter how many cores you add.

- **Implication:** To achieve significant overall performance improvements, it's crucial to minimize the sequential portion of a program. Amdahl's Law highlights the challenge of achieving perfect scalability in parallel systems.

Activities: Simple Pipeline Diagram Analysis

Activity 1: Pipeline Timing Diagram

Consider a 5-stage pipeline (IF, ID, EX, MEM, WB). Assume each stage takes 1 clock cycle. Draw a timing diagram for 4 instructions (I1, I2, I3, I4) executing in this pipeline, assuming no hazards.

Clock Cycle	Stage 1 (IF)	Stage 2 (ID)	Stage 3 (EX)	Stage 4 (MEM)	Stage 5 (WB)
1	I1				
2	I2	I1			
3	I3	I2	I1		
4	I4	I3	I2	I1	
5		I4	I3	I2	I1
6			I4	I3	I2
7				I4	I3
8					I4

- How many clock cycles does it take to execute 4 instructions?
- How many instructions are completed per clock cycle after the pipeline is full?

Activity 2: Data Hazard Example

Consider the following two instructions: I1: ADD R1, R2, R3 (R1 ← R2 + R3) I2: SUB R4, R1, R5 (R4 ← R1 - R5)

1. Identify the data dependency between I1 and I2.
2. In a 5-stage pipeline, in which stage would I2 typically be when I1 is in its WB stage?

3. Explain how **forwarding/bypassing** would help resolve this data hazard.

Activity 3: Amdahl's Law Calculation

A program takes 100 seconds to execute. 80% of the program's execution time can be perfectly parallelized.

1. What is the maximum theoretical speedup you can achieve by using an infinite number of processors?
2. If you use 4 processors, what would be the speedup?

Review Questions.

1. Explain the concept of *instruction pipelining*. How does it improve CPU performance compared to a non-pipelined processor?
2. List and briefly describe the typical stages in a five-stage instruction pipeline. What operation is performed in each stage?
3. Define the three main types of pipeline hazards. For each, give one example of how it can occur.
4. A non-pipelined processor completes one instruction every 10 ns. A pipelined version divides the work into 5 stages, each taking 2 ns.

    (a) What is the ideal speedup of the pipelined processor?

    (b) What factors might prevent achieving this ideal speedup in practice?

5. Describe any two techniques used to handle data hazards in instruction pipelines. Give an example of how each technique resolves the issue.